Pflichtenheft – Delivery-Gateway-Service (Provider Dispatcher)

Projekt:	Messaging / Communication Platform
Service:	delivery-gateway-service (Service D)
Version:	1.1 (konsolidiert & korrigiert)
Stand:	27.02.26
Technologie:	Java 21, Spring Boot 4.0.3, Maven, MariaDB/MySQL, Flyway, Docker, OpenAPI
Multi-Tenant:	ja (Tenant = companyId)

Inhaltsverzeichnis

1. Zweck und Zielbild....3
2. Begriffe (Glossar)....3
2. Scope / Nicht-Scope....3
2.1 Im Scope....3
2.2 Nicht im Scope (wird absichtlich nicht in D implementiert)....4
3. Inter-Service-Schnittstellen (klar & mit exakten Bezeichnungen)....4
3.1 Was andere Services von D erhalten (D als Anbieter)....4
3.2 Was D von anderen Services/Systemen holt (D als Konsument)....5
4. Sicherheits- und Multi-Tenant-Festlegungen....5
4.1 Authentifizierung / Autorisierung....5
4.2 Provider Callback Security....5
5. Statusmodell....6
5.1 Attempt-Status....6
5.2 Event-Typen (Timeline)....6
6. API-Verträge (Request/Response)....6
6.1 POST /api/v1/deliveries – DeliveryAttemptRequest....6
6.2 GET /api/v1/deliveries/{attemptId} – AttemptStatusResponse....7
6.3 POST /api/v1/providers/{provider}/events – ProviderEvent....8
6.4 Idempotenz (praktisch, implementierbar)....8
7. Datenhaltung (MariaDB/MySQL)....8
7.1 Tabellen....8
7.2 Wesentliche Felder....9
7.3 Indizes / Constraints....10
8. Verarbeitung / Ablauf (intern)....10
8.1 Synchronous oder Asynchronous....10
8.2 Fehlerklassifikation....10
9. Konfiguration (application.properties + ENV)....10
9.1 Prinzip....10
9.2 ENV-Variablen mit Defaults....11
10. Observability / Fehlerantworten....11
11. Dokumentation & Betriebsfähigkeit....12
12. Implementierungs-Checkliste 12

1. Zweck und Zielbild

Der **Delivery-Gateway-Service** ist die **technische Ausführungsschicht** für externe Zustellung:

- nimmt **konkrete Zustellversuche** (Delivery Attempts) per REST entgegen,
- persistiert Attempt + Timeline (Events),
- führt die Zustellung über Provider/Adapter aus (z. B. SMTP, SMS/WhatsApp Provider),
- nimmt Provider-Callbacks entgegen, mappt sie auf interne Event-Typen und aktualisiert den Attempt-Status,
- stellt Status + Timeline für Support/Folgeprozesse bereit.

Wichtig (Abgrenzung): Der Service ist **kein Orchestrator**. Er entscheidet nicht über Policies, Reihenfolge, Fallbacks oder "welcher Kanal als nächstes". Das liegt beim **aufrufenden Orchestrator-Service** (z. B. `messaging-service` oder `communication-service`, je nach Plattform-Setup).

2. Begriffe (Glossar)

- **Attempt (DeliveryAttempt):** ein konkreter Zustellversuch für einen Kanal (EMAIL/SMS/WHATSAPP/VOICE) inkl. Empfänger(n) und Payload.
- **Event (DeliveryEvent):** zeitlich geordnetes Zustellereignis (QUEUED, SENT, DELIVERED, FAILED …) als Timeline.
- **ProviderMessageId:** ID des Providers für spätere Callback-Zuordnung.
- **Idempotency-Key:** Header zur Wiederholungssicherheit (kein Doppelversand bei Retries).
- **Tenant / companyId:** Mandant. Alle Datenzugriffe sind strikt an `companyId` gebunden.
- **CorrelationId:** Request-Korrelation (Header `X-Correlation-Id`) für verteiltes Logging/Tracing.

2. Scope / Nicht-Scope

2.1 Im Scope

- REST API für Attempt-Annahme, Statusabfrage, Provider-Callbacks
- Persistenz in MariaDB/MySQL (Attempts, Events, optional Outbox)
- Provider-Adapter (SMTP, SMS/WhatsApp Provider)
- Mapping Provider-Event → internes Statusmodell
- Multi-Tenant-Isolation (`companyId` aus JWT)

2.2 Nicht im Scope (wird absichtlich nicht in D implementiert)

- **Orchestrierung/Policy/Fallback/strategische Retries** (liegt beim Orchestrator-Service)
- Inbox/In-App Zustellung
- Webhook-Delivery-Engine für Fremdsysteme (liegt bei einem dedizierten Webhook-Service)
- Template-Rendering / Content-Composition
- Eigenes File-Storage

3. Inter-Service-Schnittstellen (klar & mit exakten Bezeichnungen)

3.1 Was andere Services von D erhalten (D als Anbieter)

(A) Orchestrator-Service → Delivery-Gateway (Attempts & Status)

- `POST /api/v1/deliveries`
  Legt einen Attempt an (und startet ggf. die Zustellung).
  Response: `202` mit `DeliveryReceipt` (inkl. `attemptId`)
- `GET /api/v1/deliveries/{attemptId}`
  Liefert Attempt + Timeline (Events) für Support/Weiterverarbeitung.
  Response: `200` mit `AttemptStatusResponse`
- Optional (nur wenn du diese Steuerung wirklich in D haben willst; Standard ist "Retry via neuem Attempt durch Orchestrator"):
  - `POST /api/v1/deliveries/{attemptId}/retry`
  - `POST /api/v1/deliveries/{attemptId}/cancel`

(B) Provider → Delivery-Gateway (Callbacks)

- `POST /api/v1/providers/{provider}/events`
  Nimmt Provider-Statusereignisse entgegen und persistiert sie als DeliveryEvent.
  Response: `202` mit `DeliveryEventReceipt` (oder minimalem Ack)

(C) Betriebsendpunkte (Ops/Monitoring)

- `GET /actuator/health`
- `GET /actuator/info`
- `GET /swagger-ui`
- `GET /v3/api-docs`

3.2 Was D von anderen Services/Systemen holt (D als Konsument)

(A) Authorization Server (JWT/JWKS)

- D validiert JWTs über die konfigurierte JWKS-URL:
  ENV/Property: `JWT_JWK_SET_URI` → `spring.security.oauth2.resourceserver.jwt.jwk-set-uri`

(B) Externe Provider

- **EMAIL:** SMTP Server (Host/Port/Username/Password)
- **SMS/WHATSAPP:** Provider HTTP APIs über Adapter (z. B. Twilio/Vonage/Meta)

(C) Attachments (zwei zulässige Modi; beides ist implementierbar)

1. **Inline (klein):** Attachment kommt als Base64 im Attempt-Request → D muss nichts extern laden.
2. **Referenz (skalierbar):** Attachment kommt als `fileId` oder `url` → D lädt beim Versand:
   - `GET /api/v1/files/{fileId}/download` *(falls du einen File-Service so benennst)*
   - oder direkte `attachments[].url` (pre-signed URL, z. B. S3/MinIO)

Wenn du "später sauber skalieren" willst: **Referenzmodus** ist der Standard, Inline bleibt optional für kleine Dateien.

(D) Interne Push-Weiterleitung (bewusst nicht fest verdrahtet in D)

- D ruft **keine** internen Services "ungefragt" zurück, solange kein dedizierter Webhook-Delivery-Service als Ziel festgelegt ist.
- Status/Events werden primär über **Pull** konsumiert (`GET /api/v1/deliveries/{attemptId}`).

4. Sicherheits- und Multi-Tenant-Festlegungen

4.1 Authentifizierung / Autorisierung

- Alle fachlichen Endpunkte sind JWT-geschützt (außer Health/Info/Docs).
- Tenant wird aus JWT-Claim **`companyId`** abgeleitet.
- Jeder DB-Zugriff wird stets mit Filter `companyId = <claim>` ausgeführt.
- Cross-Tenant Zugriff ergibt 404/403 (keine Informationslecks).

4.2 Provider Callback Security

- Provider-Callbacks werden je Provider abgesichert (Signatur/HMAC Header oder vergleichbar).
- Mapping "welcher Provider nutzt welche Signatur" ist in Adapter-Konfiguration dokumentiert.

5. Statusmodell

5.1 Attempt-Status

- `QUEUED` – angenommen, wartet auf Verarbeitung
- `PROCESSING` – Versand läuft
- `SENT` – Provider hat angenommen/gesendet (nicht zwingend zugestellt)
- `DELIVERED` – zugestellt bestätigt (wenn Provider das liefern kann)
- `FAILED` – final fehlgeschlagen
- `CANCELED` – abgebrochen

5.2 Event-Typen (Timeline)

Mindestens:

- `QUEUED`, `PROCESSING`, `SENT`, `DELIVERED`, `FAILED`, `CANCELED`

Optional, wenn sauber definierbar:

- `BOUNCED`, `REJECTED`, `UNDELIVERABLE`, `READ`

6. API-Verträge (Request/Response)

6.1 POST `/api/v1/deliveries` – DeliveryAttemptRequest

Header

- `Authorization: Bearer <JWT>`
- `X-Correlation-Id` (optional; wenn fehlt, wird generiert)
- `Idempotency-Key` (empfohlen; siehe 6.4)

Body (Schema)

```
{
  "attemptId": "att-123",
  "channel": "EMAIL",
  "provider": "SMTP",
  "from": "myName",
  "to": "user1@example.com;user2@example.com",
  "deliveryMode": "SINGLE",
  "subject": "Ein Subject",
  "content": {
    "text": "Text",
    "html": "<p>Text</p>"
  },
  "attachments": [
    {
      "filename": "file.pdf",
      "mimeType": "application/pdf",
      "size": 1234,
      "base64": "....",
      "fileId": "f-123",
      "url": "https://..."
```

```
    }
  ],
  "meta": {
    "clientRef": "optional",
    "tags": ["optional"]
  }
}
```

Kanalregeln (fachlich)

- SMS/WHATSAPP: `subject` wird **ignoriert** (nicht an Provider übergeben).
- EMAIL: `subject` wird genutzt.
- Mehrere Empfänger sind zulässig (Trenner `,` oder `;`).

Response 202 – DeliveryReceipt

```
{
  "attemptId": "att-123",
  "status": "QUEUED",
  "providerMessageId": null,
  "createdAtUtc": "2026-02-27T12:00:00.000Z",
  "correlationId": "corr-..."
}
```

6.2 GET `/api/v1/deliveries/{attemptId}` – AttemptStatusResponse

Response 200

```
{
  "attempt": {
    "attemptId": "att-123",
    "companyId": "c-1",
    "channel": "SMS",
    "provider": "TWILIO",
    "to": ["+4917...","+4916..."],
    "status": "SENT",
    "providerMessageId": "SMxxxx",
    "createdAtUtc": "2026-02-27T12:00:00.000Z",
    "updatedAtUtc": "2026-02-27T12:00:10.000Z"
  },
  "events": [
    {
      "eventType": "QUEUED",
      "eventAtUtc": "2026-02-27T12:00:00.000Z",
      "rawStatus": "accepted",
      "providerMessageId": null,
      "errorCode": null,
      "errorMessage": null
    },
    {
      "eventType": "SENT",
      "eventAtUtc": "2026-02-27T12:00:05.000Z",
      "rawStatus": "queued",
      "providerMessageId": "SMxxxx",
      "errorCode": null,
      "errorMessage": null
    }
  ]
}
```

6.3 POST `/api/v1/providers/{provider}/events` – ProviderEvent

Provider schickt sein Event. D normalisiert und persistiert als DeliveryEvent.

Zuordnung Attempt

- bevorzugt über `providerMessageId`
- alternativ über `attemptId` (falls Provider/Integrator es mitsendet)

Response 202

```
{
  "accepted": true,
  "provider": "twilio",
  "receivedAtUtc": "2026-02-27T12:00:20.000Z"
}
```

6.4 Idempotenz (praktisch, implementierbar)

Es gibt zwei Ebenen (beides kompatibel):

1. **Header Idempotency-Key (empfohlen)**
    - Unique: `(companyId, idempotencyKey)`
    - Wiederholung liefert denselben Receipt, ohne zweiten Versand.
2. **attemptId (wenn vom Orchestrator vorgegeben)**
    - Unique: `(companyId, attemptId)`
    - Wiederholung liefert denselben Receipt.

> Implementationshinweis: du setzt **mindestens eine** Unique-Regel in der DB. Viele Teams setzen beide, wobei `Idempotency-Key` als "Retry-Schutz" der bessere Standard ist.

7. Datenhaltung (MariaDB/MySQL)

7.1 Tabellen

- `delivery_attempt`
- `delivery_event`
- optional `outbox_event` (wenn Worker/Outbox Pattern genutzt wird)

7.2 Wesentliche Felder

delivery_attempt

- `id` (PK)
- `company_id` (Tenant)
- `attempt_id`
- `idempotency_key` (nullable)
- `channel`
- `provider`
- `from_value`
- `to_value` (raw) + optional normalisierte Liste/JSON
- `subject` (nullable)
- `content_text`, `content_html` (nullable)
- `attachments_json` (nullable)
- `status`
- `provider_message_id` (nullable)
- `error_code`, `error_message` (nullable)
- `correlation_id`
- `created_at_utc`, `updated_at_utc`

delivery_event

- `id` (PK)
- `company_id`
- `attempt_id`
- `provider`
- `channel`
- `event_type`
- `event_at_utc`
- `raw_status`
- `provider_message_id` (nullable)
- `error_code`, `error_message` (nullable)
- `raw_payload_json` (optional, wenn erlaubt)
- `created_at_utc`

7.3 Indizes / Constraints

- Unique (mindestens eins):
  - `UNIQUE(company_id, attempt_id)` **oder**
  - `UNIQUE(company_id, idempotency_key)`
- Index:
  - `(company_id, status)`
  - `(company_id, attempt_id)`
  - `provider_message_id` (für Callback-Join)
  - `(attempt_id, event_at_utc)` (Timeline)

8. Verarbeitung / Ablauf (intern)

8.1 Synchronous oder Asynchronous

- D kann Versand **synchron** (direkt im Request) oder **asynchron** (Worker) ausführen.
- Bei asynchron:
  - Attempt wird initial `QUEUED`,
  - Worker setzt `PROCESSING` und schreibt Events,
  - final `SENT/DELIVERED/FAILED`.

8.2 Fehlerklassifikation

- `transient/retryable` (Timeout, Netzwerk, Provider 5xx)
- `non-retryable` (ungültiger Empfänger, 4xx, Policy Violation)
- Strategische Retries/Fallbacks bleiben beim Orchestrator; D kann höchstens kurze technische Retries machen (1–2), und protokolliert jeden Versuch als Event.

9. Konfiguration (application.properties + ENV)

9.1 Prinzip

- `application.properties` enthält Defaults (Dev/Local), keine Secrets.
- ENV überschreibt Defaults.

Beispiel `application.properties` (Kern)

```
server.port=${SERVER_PORT:8106}
spring.datasource.url=jdbc:mariadb://${DB_HOST:localhost}:${DB_PORT:3306}/$
{DB_NAME:deliveries}
spring.datasource.username=${DB_USER:root}
spring.datasource.password=${DB_PASSWORD:}
spring.security.oauth2.resourceserver.jwt.jwk-set-uri=$
{JWT_JWK_SET_URI:http://localhost:9000/oauth2/jwks}
```

9.2 ENV-Variablen mit Defaults

ENV	Default	Bedeutung
`SERVER_PORT`	`8106`	Service-Port
`DB_HOST`	`localhost`	DB Host
`DB_PORT`	`3306`	DB Port
`DB_NAME`	`deliveries`	DB Name
`DB_USER`	`root`	DB User
`DB_PASSWORD`	*(leer)*	DB Passwort
`JWT_JWK_SET_URI`	`http://localhost:9000/oauth2/jwks`	JWKS URL

Provider (optional, je nach Adapter-Implementierung)

ENV	Default	Bedeutung
`EMAIL_HOST`	`smtp.gmail.com`	SMTP Host
`EMAIL_PORT`	`587`	SMTP Port
`EMAIL_USERNAME`	*(leer)*	SMTP User
`EMAIL_PASSWORD`	*(leer)*	SMTP Pass (als Secret)
`EMAIL_AUTH`	`true`	SMTP Auth
`EMAIL_STARTTLS`	`true`	STARTTLS
`TWILIO_ACCOUNT_SID`	*(leer)*	Provider Credential (Secret)
`TWILIO_AUTH_TOKEN`	*(leer)*	Provider Credential (Secret)

10. Observability / Fehlerantworten

- `X-Correlation-Id` wird übernommen oder generiert und in Response zurückgegeben.
- Strukturierte Logs mit `companyId`, `attemptId`, `correlationId`.
- Standard-ErrorResponse (JSON) enthält mindestens: `timestampUtc`, `status`, `error`, `message`, `path`, `correlationId`.

11. Dokumentation & Betriebsfähigkeit

- Swagger UI unter `/swagger-ui`, OpenAPI unter `/v3/api-docs`
- README enthält:
  - Setup & ENV
  - Start local (mvn, docker compose)
  - Beispiel-curls:
    - `POST /api/v1/deliveries`
    - `GET /api/v1/deliveries/{attemptId}`
    - `POST /api/v1/providers/{provider}/events`

12. Implementierungs-Checkliste

1. Port in Code/Compose/Dockerfile auf **8106** (Default + EXPOSE + Mapping)
2. Endpoint-Namen konsolidieren auf:
   - `/api/v1/deliveries`
   - `/api/v1/providers/{provider}/events`
     (optional Alias für alte Callback-Route, falls vorhanden)
3. Statusmodell in Code auf `QUEUED/PROCESSING/SENT/DELIVERED/FAILED/CANCELED` vereinheitlichen
4. DB: Unique Constraint für Idempotenz (mindestens `(company_id, attempt_id)` oder `(company_id, idempotency_key)`)
5. Multi-Tenant Filter: **jede** Query/Update/Delete Operation scoped mit `company_id`
6. Subject-Regel: SMS/WhatsApp **ignorieren**, Email nutzen
7. Provider Callback Mapping + Signaturprüfung pro Provider
8. GET Status + Timeline implementieren (Attempt + Events sortiert)